



2+

FACILITATOR BRIEFING —

Read the night before. About 20 minutes.

GitHub 101: What You Need to Know to Teach It

WHAT THIS IS —

Everything behind the 45-minute plan: the concepts in plain terms, your demo click by click on your own account, the questions the room is likely to ask with answers, and the tips worth handing out. The plan is what you run; this is what you know while running it.

WHAT THE ROOM NEEDS —

Not Git. The room needs the idea that work can have a history, that the history can be shared without being exposed, and that nothing committed is ever really lost. Everything else (terminals, rebasing, Actions) can wait until someone has a reason to want it.

The model in one page CONCEPTS

Git is a free program, written in 2005 to manage the Linux source code, that records snapshots of a folder. You tell it when to take a snapshot (a **commit**) and you write one line saying what changed. It keeps every snapshot forever, so you can compare any two versions or return to any of them. Git runs on your own machine and needs no internet.

GitHub is a website, owned by Microsoft since 2018, that hosts copies of Git folders and adds the social layer: a web page for every project, review of proposed changes, a to-do tracker, and access control. GitLab and Bitbucket do the same job. When people say "put it on GitHub" they mean: keep a Git history, and keep a copy of it on GitHub so others (or your other laptop) can reach it.

The one idea that makes the rest click: a Git history is a **chain of labeled snapshots**, and a **branch** is just a second chain that splits off from the first so you can try something without touching the main version. When the experiment works, you **merge** it back. On GitHub, the polite way to merge is a **pull request**: a page that shows exactly what the branch changes, line by line, with room for comments, and a button to merge once someone has read it. Teams use pull requests to review each other; a solo researcher uses them to review an AI agent's changes, or their own work from a tired Friday.

TERM	WHAT IT MEANS	WHERE TO POINT ON YOUR DEMO REPOSITORY
Repository (repo)	A project folder plus its full history	llm-revealed-preferences itself
Commit	A snapshot with a message and an author	The "74 commits" link
Diff	The line-by-line difference between two versions; red removed, green added	Click any commit

TERM	WHAT IT MEANS	WHERE TO POINT ON YOUR DEMO REPOSITORY
README	A text file GitHub renders as the project's front page	Scroll below the file list
Branch; main	A parallel line of work; main is the default one (called master before 2020)	The branch dropdown, top left
Merge	Folding one branch's changes into another	Shown live in the practice repo
Pull request (PR)	A proposal to merge, with the diff and a comment thread	Pull requests tab
Merge conflict	Two branches changed the same lines; Git asks you to choose	The "break on purpose" exercise
Clone	A full copy of a repo on your computer, linked to the original	Green "Code" button
Push; pull	Send your new commits up to GitHub; bring others' commits down	n/a in the browser; GitHub Desktop has buttons for both
Remote; origin	The copy on GitHub; origin is its default nickname	n/a
Fork	Your own GitHub copy of someone else's repo, used to propose changes to it	"Fork" button, top right
Issue	A numbered to-do or bug report attached to the repo	Issues tab
.gitignore	A list of files Git should never track	Keeps <code>.env</code> and <code>literature/</code> out
License	The file that says what others may do with the work; without one, nobody may legally reuse it	Your demo repo has none; worth saying
Release; tag	A named, frozen version (v1.0, "draft to committee")	Releases, right sidebar
github.dev	A full editor in the browser; press <code>.</code> on any repo	Do it live
Codespaces	A cloud computer with the repo loaded, so code runs without installing anything; a monthly free allowance	"Code" button, Codespaces tab
Actions	Scripts GitHub runs automatically on each push (tests, builds)	Actions tab; skip tomorrow
Pages	Free website hosting from a repo	Settings, Pages; skip tomorrow
Profile README	A repo named after your username; its README shows on your profile	Your jblakeslee89 repo

Three ways in RECOMMEND THE FIRST TWO

WAY	GOOD FOR	COST
The browser	Creating repos, editing text, reviewing and merging pull requests, reading history. Everything tomorrow.	Awkward for many files at once; can't run code (unless in a Codespace).
GitHub Desktop	Working on files in a real folder on your laptop, with buttons for commit, push, and pull. The right next step for most of the room.	One install; macOS and Windows only.
Terminal, or an agent	Full control. An agent such as Claude Code runs the Git commands for you and shows you what it did.	Steep for newcomers; an agent's Git commands still need reading.

If someone insists on the terminal, these six cover a month of solo work:

```
git clone https://github.com/USER/REPO.git # copy a repo to this computer
git status # what has changed since the last commit
git add -A # stage every change for the next commit
git commit -m "Say what changed and why" # take the snapshot
git pull # bring down others' commits
git push # send yours up
```

Your demo, click by click ON YOUR OWN ACCOUNT

0

SETUP

BEFORE YOU SHARE

Open `github.com/jblakeslee89/llm-revealed-preferences` in its own tab and share that tab only. Your home page and repository sidebar list your private repositories, which the room doesn't need to see. Sign out in a second browser window once tonight and look at your profile as the room will: six public repositories and your profile README.

1

MIN

THE FRONT PAGE

The file list, then the README below it. Say: the README is the first thing anyone reads, including you in six months. Point at the description line and the language bar (Python).

2

MIN

THE HISTORY

Click "74 commits." Read three messages aloud, for instance "Phase 4: save the adapter at every checkpoint." Every one is a version you could restore. Open one and scroll to the bottom of the message: `Co-Authored-By` names the model. All 74 carry it. That's attribution recorded on every change, which is Session 1's rule in practice.

2

MIN

A DIFF

In that commit, show red and green lines. Say: this is what you read before accepting anything an agent writes. Then open a single file and click History to show the same idea for one file.

2

MIN

WHAT STAYS OFF GITHUB

Open `.gitignore`. Point at `.env` (the API key) and `literature/` with its comment "copyrighted, keep local." Then open `.env.example`: it says where the key goes and holds no key. This is the single most useful pattern in the session.

1

MIN

THE EDITOR TRICK

Press `.` with the repository open. The whole project opens in an editor in the browser. Close it; nothing changed unless you commit.

1

MIN

YOUR PROFILE

Optional. Open your profile and show the README from the `jblakeslee89` repository: a one-paragraph bio and links to current work. For anyone job-hunting, this is the page people land on.

Questions they may ask WITH ANSWERS

THE BASICS —

What's the difference between Git and GitHub?	Git is the free program that keeps the history, on your computer. GitHub is a website that hosts Git repositories and adds review, to-do tracking, and sharing. You can use Git without GitHub; you can't really use GitHub without Git, though the browser hides it.
Do I need to know how to code?	No. Git tracks any files, and works best with plain text: Markdown notes, LaTeX, CSVs, prompts, skill docs. This group's prompts and skill docs are ideal first contents.
Can I use it for Word documents or PDFs?	You can store them and keep versions, but GitHub can't show what changed inside them; it just says the file changed. For prose you want to track, write in Markdown or LaTeX. Overleaf can sync a LaTeX project with GitHub on paid and institutional plans.
How is this different from Google Drive or OneDrive?	Drive saves constantly and keeps versions nobody labels. Git saves only when you commit, and every save carries a reason. Git also lets you run parallel drafts (branches) and merge them. Drive is still better for several people editing one prose document at the same time.
How often should I commit?	Whenever you reach a state you'd want to return to: a finished step, a working script, a completed section. Small and often beats one giant commit on Friday.
What should a commit message say?	One line, starting with a verb, saying what changed and why if it isn't obvious. "Drop 2019 rows counted twice in the merge," not "update."
What's the difference between a fork, a clone, and a branch?	A branch is a parallel line of work inside one repository. A clone is a copy of a repository on your computer. A fork is your own copy of someone else's repository on GitHub, which you use to propose changes when you can't edit theirs directly.
What is a merge conflict, and is it bad?	Two versions changed the same lines, and Git won't guess which to keep. It marks both versions in the file and asks you to choose. It's routine, and GitHub has a web editor for simple ones. The practice exercise creates one on purpose.
What does "main" mean? I've seen "master."	The name of the default branch. GitHub changed the default from master to main in 2020; older repositories still use master. They behave the same.
PRIVACY AND SAFETY —	
Is everything I put on GitHub public?	No. You choose for each repository, and private repositories are free and unlimited. Your profile and public repositories are visible to anyone. You can invite specific people to a private repository.
I pushed a password or API key. What do I do?	Revoke the key first, at whichever service issued it, and make a new one. Assume it's already been copied: automated scanners search public GitHub constantly. Deleting the file in a new commit doesn't help, because the key is still in the history. GitHub now blocks many recognizable key formats before they reach public repositories, but don't count on it. The fix is the <code>.env</code> pattern from the demo.
Can I put data in a repository?	Small public or synthetic data, yes. Anything sensitive, personally identifying, proprietary, licensed from a vendor, or covered by a RAND data agreement: no, not even in a private repository, unless the project says otherwise. GitHub refuses single files over 100 MB and warns above 50 MB; large files need Git LFS, and most research data should stay out anyway.

If I delete a repository, is it gone?	From GitHub, yes. But anyone who cloned or forked a public repository still has a copy, and public content may already be cached elsewhere. Treat anything you ever made public as permanent.
Should I turn on two-factor authentication?	Yes. GitHub now requires it of anyone who contributes code, and prompts for it during setup. An authenticator app is better than text messages.

RAND, RESEARCH, AND CAREERS —

Can I put RAND project code in my personal account?	Don't answer this from opinion. RAND has its own organization at github.com/RANDCorporation , and project work belongs to the project. Ask the project lead, and log it in the Playbook as a question to raise upward. Personal accounts are for your own coursework, dissertation practice, and anything you'd be comfortable seeing in public.
How does this help with my dissertation?	Three ways. A history of how the analysis evolved, so you can answer "what changed since the last draft?" exactly. A frozen release for each draft you send to your committee. And a replication package: many journals now expect one, and a clean repository with a README is most of the work. Connecting a repository to Zenodo gives each release a citable DOI, and a <code>CITATION.cff</code> file adds a "Cite this repository" button.
Does a GitHub profile matter for jobs?	For quantitative and technical roles, yes. People read pinned repositories and their READMEs, not the green activity squares. Three well-documented projects beat fifty empty ones. A profile README (a repository named after your username) is the landing page.
Which license should I use?	Without a license, the default is full copyright: people can look but not legally reuse. MIT is the common permissive choice for code; CC BY 4.0 is common for text and data. choosealicense.com walks through it. For anything touching RAND work, check RAND's rules first.
Is there a student discount?	The GitHub Student Developer Pack, through GitHub Education, gives verified students free Copilot and GitHub Pro among other tools. Verification needs proof of enrollment; a school email address usually makes it quick.

THE MODEL AND GITHUB —

Can Claude or ChatGPT read my repositories?	Most frontier tools can connect to GitHub and read files from a repository you choose; coding agents such as Claude Code, Codex, and Copilot's agent work inside a repository directly and can open pull requests. Whether RAND's approved tool may connect to GitHub is a policy question; check before connecting anything with project material in it.
Why does GitHub matter more now that agents write code?	Because an agent can change twenty files in a minute. With a commit history, every one of those changes is a diff you can read and a version you can restore. Without one, you're trusting the agent's summary of what it did. It's Session 3's "read the code" rule, made structural.
Should I let the model write my commit messages?	Fine, if you read them. The model can see what changed; it often can't know why. Add the why yourself.
Where do skill docs live?	Often in repositories. Anthropic's public skills repository (github.com/anthropics/skills , linked in Session 5) is an example. A repository is a natural way to share the group's skill docs with version history.

It won't accept my password when I push.	GitHub stopped accepting account passwords for Git in 2021. Use GitHub Desktop, which signs in through the browser, or the GitHub CLI (<code>gh auth login</code>). Personal access tokens and SSH keys also work but take longer to explain.
"Updates were rejected because the remote contains work that you do not have."	Someone (or you, from another machine or the browser) committed to GitHub since you last pulled. Pull first, resolve anything that conflicts, then push. Don't reach for force-push; it overwrites their work.
I think I broke everything.	If it was committed, it isn't lost; every earlier version is still there. The only work truly at risk is uncommitted work. Paste the exact error into the Git-tutor prompt. Prefer <code>git revert</code> , which undoes a commit by adding a new one, over anything with "reset" or "force" in it.

Tips to give them WITH THE REASON

TIP	WHY
Private by default	Making something public is easy and permanent. Do it on purpose.
Write the README first	It forces you to say what the project is, and it's what future readers, including you, see first.
Set up .gitignore before the first commit	Anything committed once stays in the history. Keys, data, and readings should never enter it.
Keys in .env; commit .env.example	Others learn what the code needs without seeing your key.
Messages say why	The diff already shows what changed. The reason is the part only you know.
Pull before you start, push when you stop	Prevents most conflicts, including the ones with yourself on another laptop.
Prefer plain text	Markdown, LaTeX, CSV, and code show their changes. Word and PDF don't.
One project, one repository	Histories stay readable, and you can share one project without the others.
Tag a release at each milestone	"Draft to committee" becomes a version you can always return to exactly.
Use a pull request even alone	It makes you read the full diff before merging, which is the habit that catches agent mistakes.
Read other people's repositories	A good README or folder layout is easier to copy than to invent. Star the ones worth returning to.
Use the model as a Git tutor	Paste the exact error, ask for the smallest safe fix, and forbid force-push and reset --hard.

- CHECK BEFORE TOMORROW —
- Send the note so accounts exist before the session starts.
 - Find RAND's current guidance on code in personal accounts and on the RANDCorporation organization, so you can point to it rather than paraphrase it.
 - Find out whether RAND's approved AI tool may connect to GitHub.
 - View your profile signed out, and confirm the demo repository shows nothing you'd rather not share.
 - Do the practice exercise once yourself, including the merge conflict, so the clicks are familiar.